



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Ingeniería en Computación

Ingeniería de Software

Grupo 4

Ing. Orlando Zaldivar Zamorategui

Proyecto

Fundamentos de los requerimientos de software.

Definición de un requerimiento de software

Equipo 1

321101464

Castañeda González Michelle Ariana

321009700

Martínez Gil Samantha Abigail

321316260

Martínez Martínez Camila Danae

424136666

Hernandez Moguel Francisco

321046309

Mora Celis Ángel Gabriel

321041878

Olayo Ramírez Emiliano

Fecha de petición:16 de octubre de 2025.

Fecha de entrega:19 de noviembre de 2025

Índice

1. Objetivo.....	4
2. Introducción.....	4
3. Desarrollo.....	4
Definición de requerimiento de software.....	4
Origen de los requerimientos de software.....	5
Ingeniería de requerimientos.....	6
Características de un buen requerimiento.....	7
Propósito de los requerimientos.....	8
Importancia de la especificación de los requerimientos.....	8
Ciclo de vida de los requerimientos.....	9
1. Elicitación de los requerimientos.....	10
2. Análisis y negociación.....	10
3. Especificación de requerimientos.....	10
4. Validación de requerimientos.....	11
5. Gestión de requerimientos.....	11
Proceso de definición.....	12
Errores comunes en la definición de requerimientos.....	13
Dificultades esenciales y accidentales.....	15
Dificultades esenciales.....	15
Dificultades accidentales:.....	16
Buenas prácticas para una definición exitosa de requerimientos de software.....	18
4. Ejemplos.....	20
EJEMPLO 1 — Sistema hospitalario (Registro de pacientes).....	20
EJEMPLO 2 — Comercio electrónico (Proceso de pago en línea).....	21
EJEMPLO 3 — Aplicación universitaria (Consulta de calificaciones).....	21
EJEMPLO 4 — Banca en línea (Autenticación de usuarios).....	22
EJEMPLO 5 — Gobierno digital (Gestión de trámites).....	23
EJEMPLO 6 — Salud pública (Control de vacunación).....	23
EJEMPLO 7 — Transporte urbano (Gestión de rutas).....	24
EJEMPLO 8 — Manufactura (Control de inventarios).....	25
EJEMPLO 9 — Redes sociales (Publicaciones y privacidad).....	25
EJEMPLO 10 — Energía (Medición inteligente).....	26
5. Video.....	27
6. Software interactivo.....	27
7. Cuestionario interactivo.....	27
8. Conclusión.....	27
9. Bibliografía.....	28

2.2.1. Definición de un requerimiento de software

1. Objetivo

El objetivo del trabajo es proporcionar un material accesible y didáctico que facilite la comprensión del concepto de requerimientos de software, así como su importancia al desarrollar proyectos de ingeniería de software, esto con el fin de garantizar el buen funcionamiento del programa, la calidad y la satisfacción del cliente.

Tal que al final, el lector sea capaz de identificar y definir los requerimientos de software de manera efectiva, comprendiendo su relevancia al momento de desarrollar un proyecto de software exitoso.

2. Introducción

El desarrollo de software constituye una actividad compleja que demanda precisión en la interpretación de las necesidades humanas y organizacionales. Dentro de este proceso, los requerimientos de software representan el cimiento sobre el cual se construyen los sistemas informáticos.

Como señala Pressman (2010), los requerimientos representan el vínculo entre la intención del cliente y la solución técnica del ingeniero, estableciendo así la base de todo el proceso de desarrollo. Si esta fase se ejecuta incorrectamente, los errores se propagan a las etapas posteriores, generando sobrecostos y pérdida de calidad. En consecuencia, la ingeniería de requerimientos surge como una disciplina clave para definir, documentar y validar las necesidades del sistema antes de su implementación.

Los requerimientos constituyen, por tanto, un puente entre la necesidad y la solución. Su análisis no solo involucra aspectos técnicos, sino también humanos, comunicativos y organizacionales. Por esta razón, el estudio de su definición, tipología y gestión reviste un carácter fundamental dentro de la ingeniería de software contemporánea.

3. Desarrollo

Definición de requerimiento de software

Un requerimiento de software puede entenderse como una condición o capacidad que el sistema debe poseer para cumplir con un contrato, norma o necesidad específica. Pressman y Maxim (2020, p. 134) indican que los requerimientos funcionan como un contrato entre el cliente y el desarrollador, estableciendo el alcance y los límites del proyecto.

De acuerdo con Sommerville (2011, p. 103), estos requerimientos comprenden tanto las funciones que debe desempeñar el sistema como las restricciones que delimitan su comportamiento. La definición precisa de los requerimientos constituye un proceso comunicativo y técnico que requiere claridad, coherencia y verificabilidad.

“La formulación de requerimientos debe ser un proceso de descubrimiento y negociación más que una mera descripción de necesidades” (Sommerville, 2011, p. 104).

Esta afirmación pone de manifiesto la importancia del diálogo constante entre los usuarios, analistas y desarrolladores para garantizar que el producto final satisfaga las necesidades reales.

Origen de los requerimientos de software

Respecto al origen de los requerimientos de software, según Wong Durand, estos requerimientos surgen debido a una necesidad de automatización en el desarrollo del software, en la mayoría de los casos se describe una regla funcional a alto nivel y se describe una idea de manera global.

Así mismo, Wong nos presenta al International Institute of Business Analysis [IIBA] (2009), el cual define el término requerimiento o requisito de la siguiente forma:

The term “requirement” is one that generates a lot of discussion within the business analysis community. Many of these debates focus on what should or should not be considered a requirement, and what are the necessary characteristics of a requirement. When reading the BABOK® Guide, however, it is vital that “requirement” be understood in the broadest possible sense. Requirements include, but are not limited to, past, present, and future conditions or capabilities in an enterprise and descriptions of organizational structures, roles, processes, policies, rules, and information systems. A requirement may describe the current or the future state of any aspect of the enterprise. (p. 005).

(Wong, 2017, p.51)

Este texto explica que el término “**requirement**” (**requerimiento**) genera mucha discusión entre los analistas de negocios, porque no siempre hay acuerdo sobre qué cosas deben considerarse un requerimiento o qué características debe tener dicho requerimiento.

El *BABOK® Guide* (una guía internacional sobre análisis de negocios) aclara que debe entenderse el concepto de la forma más amplia posible. Es decir, un requerimiento no se limita solo al software, sino que puede incluir condiciones o capacidades pasadas, presentes o futuras de una empresa, así como descripciones de su estructura organizacional, funciones, procesos, políticas, reglas e incluso sus sistemas de información.

Resumiendo lo dicho por este Instituto, el texto nos narra que un requerimiento **puede describir cualquier aspecto del estado actual o futuro de la organización**, no solo se basa en lo que hace el sistema, sino todo lo que la empresa necesita o debe tener para funcionar correctamente a lo largo del tiempo.

Ingeniería de requerimientos

La ingeniería de requerimientos ayuda a plasmar los requisitos funcionales de los usuarios en diagramas técnicos que permitan a los analistas y programadores de software un mejor entendimiento del producto de software que tienen que desarrollar. En esta primera etapa, los diagramas de contexto y diagramas de flujo de datos son los primeros por elaborar para poder lograr un entendimiento con el usuario de lo que se requiere.

La ingeniería de requerimientos constituye la fase inicial del desarrollo de software, pues ayuda a generar las especificaciones preliminares para tal desarrollo.

En esta primera etapa pueden elaborarse incluso pruebas de concepto preliminares para evaluar la tecnología por aplicar y la factibilidad de continuar con el desarrollo del software. Igualmente, se aplican técnicas de experiencia de usuario para que el usuario en esta primera etapa conozca a alto nivel cómo quedarán sus interfaces. El riesgo en esta etapa es la indecisión del usuario, pues si aún no sabe concretamente lo que quiere o tiene mucha dificultad para tomar decisiones, esta etapa puede tomar mucho tiempo.

Pressman (2005) precisa que

La ingeniería de requisitos se lleva a cabo a través de siete funciones que en algunos casos pueden ejecutarse de manera paralela, sin embargo las mismas se adaptarán a la metodología a utilizar por la organización y a la necesidad del proyecto, la finalidad es

definir claramente la necesidad del usuario final de modo que pueda desarrollarse adecuadamente un análisis, diseño y construcción que refleje lo que el usuario solicitó (p. 157).

Así mismo, Pressman (2002, p.171), presenta la ingeniería de requisitos como respaldo a uno de los desafíos que se presentan al momento de generar y crear un software: “¿Cómo podemos asegurar que hemos especificado un sistema que recoge las necesidades del cliente y satisface sus expectativas?”, realmente para este desafío no existe una respuesta segura, sin embargo, mediante un buen proceso de ingeniería de requisitos se puede lograr la mejor solución posible.

Pressman definió que la función de esta ingeniería de requisitos, es la de validar y gestionar dichos requerimientos, para así lograr transformar dichos requisitos en un sistema operacional. Esto gracias a que entendiendo el proceso de ingeniería de requisitos, podemos facilitar el mecanismo apropiado para comprender lo que el cliente quiere, analizando sus necesidades, confirmando su viabilidad, negociando una solución razonable y especificando la solución sin ambigüedad. (2002, pp.171-172).

Características de un buen requerimiento

Un requerimiento correctamente definido debe poseer ciertas características que aseguren su calidad y utilidad. Entre ellas destacan la claridad, consistencia, completitud, verificabilidad y trazabilidad. Robertson y Robertson (2012) afirman que “un requerimiento de calidad es aquel que puede ser comprendido del mismo modo por todos los interesados y validado mediante evidencia objetiva” (p. 65).

De igual forma, Sommerville (2011) enfatiza que “la coherencia entre los requerimientos es esencial para evitar contradicciones que generen fallos en el diseño o en la implementación del sistema” (p. 83).

Estas propiedades garantizan que el documento de requerimientos sea confiable, facilite la comunicación y minimice los errores durante las fases posteriores.

Por ejemplo, la claridad implica evitar términos ambiguos o subjetivos como “rápido” o “fácil de usar”. En cambio, deben expresarse criterios cuantificables, como *“el sistema debe procesar 100 solicitudes por minuto con un tiempo de respuesta menor a dos segundos”*. Este nivel de detalle permite verificar objetivamente el cumplimiento del requerimiento.

Propósito de los requerimientos

El propósito fundamental de un requerimiento es establecer las condiciones bajo las cuales el sistema será considerado exitoso. Esto implica definir con claridad qué debe hacer el sistema, cómo debe comportarse y bajo qué limitaciones debe operar. Sommerville (2011) explica que “los requerimientos sirven como base de comunicación entre el cliente, los usuarios y los desarrolladores, reduciendo la ambigüedad y los malentendidos” (p. 103).

Desde esta perspectiva, los requerimientos actúan como un contrato comunicativo. Pressman y Maxim (2020) añaden que “cada requerimiento debe establecer criterios verificables que permitan comprobar su cumplimiento durante las pruebas y la validación del sistema” (p. 136). Esta función de control asegura que el producto final pueda evaluarse objetivamente, garantizando la satisfacción de las expectativas iniciales.

El propósito de los requerimientos, por tanto, no se limita a definir funcionalidades, sino que guía todo el proceso de desarrollo, establece límites, permite la planificación y ofrece una referencia constante para la toma de decisiones técnicas.

Importancia de la especificación de los requerimientos

La correcta definición de los requerimientos determina en gran medida el éxito o fracaso de un proyecto. Cuando no se definen adecuadamente, los proyectos suelen presentar sobrecostos, demoras o incluso cancelaciones. Cristiá (2018) cita un estudio del *Standish Group* realizado en 1994 sobre 8,000 proyectos de software, donde se halló que solo el 9% de los proyectos grandes fueron entregados dentro del tiempo y presupuesto previstos. Las causas principales del fracaso incluyeron requerimientos incompletos, expectativas no realistas y falta de compromiso del usuario.

En 1994 el Standish Group realizó un estudio sobre 8.000 proyectos de software con el objetivo de determinar el grado de éxito de cada uno de ellos. Los resultados

fueron bastante desalentadores: el 31% de los proyectos fueron cancelados antes de ser terminados, en las grandes compañías solo el 9% de los proyectos fueron entregados dentro del plazo y presupuesto predefinidos y el 16% en las pequeñas y medianas. A raíz de estos descubrimientos, un año más tarde Standish Group realizó otro estudio para determinar las causas de semejante fracaso. Los resultados fueron los siguientes:

- **Requerimientos incompletos (13,1%)**
- Falta de compromiso del usuario (12,4%)
- Falta de recursos (10,6%)
- Expectativas no realistas (9,9%)
- Falta de soporte ejecutivo (9,3%)
- Requerimientos y especificaciones cambiantes (8,7%)
- Falta de planeamiento (8,1%)
- Fin de la necesidad del sistema (7,5%) (Cristiá, 2018).

Este ejemplo demuestra que la ingeniería de requerimientos participa de manera directa en el éxito de los proyectos tecnológicos, ya que permite documentar, analizar y gestionar las expectativas desde las fases iniciales del desarrollo.

Ciclo de vida de los requerimientos

El ciclo de vida de los requerimientos abarca el conjunto de fases por las cuales atraviesan las necesidades del usuario desde su identificación inicial hasta su mantenimiento durante la operación del sistema. Este proceso es dinámico y reiterativo, pues los requerimientos pueden cambiar conforme se modifican las condiciones del entorno o las expectativas del cliente.

Como afirma Sommerville,

“La ingeniería de requerimientos implica un conjunto de actividades que incluyen la obtención, el análisis, la especificación, la validación y la gestión de los requerimientos, todas ellas interdependientes entre sí”
(2011, p. 120).

Estas etapas buscan asegurar que el producto final sea coherente con las necesidades reales del usuario, manteniendo un equilibrio entre lo técnico y lo funcional.

1. Elicitación de los requerimientos

La elicitación representa el punto de partida del ciclo de vida. En esta etapa se busca descubrir las necesidades reales de los usuarios y demás interesados mediante la observación, el diálogo y la exploración del contexto organizacional. Sommerville (2011) explica que la elicitación “consiste en recopilar información sobre lo que el sistema debe hacer a través de la comunicación directa con los interesados”

2. Análisis y negociación

Una vez obtenidos los requerimientos iniciales, se procede al análisis y negociación. Esta fase tiene como propósito depurar, clasificar y priorizar los requerimientos, eliminando ambigüedades y resolviendo conflictos entre las distintas partes interesadas.

Por ejemplo, el área de ventas puede requerir funcionalidades avanzadas que incrementen las ventas, mientras que el equipo de desarrollo debe equilibrar dichas demandas con los recursos y plazos disponibles. Este diálogo constante permite transformar los requerimientos iniciales en un conjunto estructurado, realista y validado. En consecuencia, el análisis y la negociación constituyen el filtro de calidad conceptual antes de pasar a la documentación formal.

3. Especificación de requerimientos

La etapa de **especificación** tiene como finalidad convertir los acuerdos alcanzados en un documento formal y verificable. Este documento, conocido como *Especificación de Requerimientos de Software (SRS)*, sirve como un contrato técnico entre los clientes y el equipo de desarrollo. Pressman (2010) explica que “una buena especificación debe describir las funciones, restricciones y propiedades del sistema con claridad, precisión y sin ambigüedades”

El contenido de una SRS incluye la descripción del propósito del sistema, sus funciones principales, requerimientos de desempeño, interfaces, restricciones, criterios de validación y trazabilidad. Robertson y Robertson (2012) complementan que

“una especificación efectiva no se limita a enumerar requerimientos, sino que presenta la información de forma comprensible, organizada y verificable”

4. Validación de requerimientos

La validación consiste en comprobar que los requerimientos definidos reflejen de manera fiel las necesidades reales del usuario y cumplan con los criterios de calidad establecidos. Según Sommerville (2011), “la validación garantiza que los requerimientos sean correctos, consistentes y completos antes de iniciar la etapa de diseño”

En esta fase se utilizan técnicas como las revisiones por pares, inspecciones formales, demostraciones de prototipos y pruebas de aceptación tempranas.

Por ejemplo, si durante la validación se descubre que una funcionalidad no cumple con las normas de seguridad de la organización, puede corregirse antes de invertir tiempo en su desarrollo. Este proceso no solo evalúa la coherencia técnica del documento, sino también su alineación con los objetivos estratégicos de la empresa.

5. Gestión de requerimientos

La gestión de requerimientos es la fase que mantiene la estabilidad del proyecto a lo largo del tiempo. Su objetivo es controlar los cambios, conservar la trazabilidad y asegurar la coherencia del sistema incluso después de su implementación.

Pohl (2010) señala que “la gestión de requerimientos permite manejar el cambio de manera estructurada, manteniendo la relación entre las necesidades originales y su implementación final”

Los requerimientos evolucionan conforme cambian las condiciones del negocio o las tecnologías disponibles. Sommerville (2011) advierte que “la gestión no termina con la entrega del documento, sino que debe acompañar todo el ciclo de vida del software”

En conjunto, las fases del ciclo de vida de los requerimientos conforman un proceso estructurado que equilibra el análisis técnico con la comprensión humana de las necesidades del usuario. Como concluye Sommerville (2011), “una gestión adecuada del ciclo de vida no solo mejora la eficiencia del desarrollo, sino que incrementa la satisfacción del cliente y prolonga la vida útil del sistema”.

Proceso de definición

El proceso de definición de requerimientos comprende una serie de actividades interdependientes que permiten transformar las necesidades del usuario en especificaciones formales. Entre estas actividades destacan la identificación de necesidades, el análisis de

viabilidad, la documentación formal y la validación inicial, las cuales se desarrollan de manera iterativa a lo largo del ciclo de vida del proyecto.

Sommerville (2011) describe este proceso como

“un ciclo iterativo de descubrimiento, documentación y validación, en el cual cada requerimiento se refina progresivamente hasta alcanzar la precisión deseada”
(p. 120).

Esta definición pone de relieve que la formulación de requerimientos no es una tarea lineal ni estática, sino un ejercicio continuo de comprensión y ajuste. En cada iteración, los analistas profundizan en el entendimiento de las necesidades del usuario y en las restricciones del entorno, lo que permite incrementar la calidad del producto final.

Durante esta fase, los analistas aplican diversas técnicas orientadas a capturar y formalizar la información de manera sistemática. Se utilizan métodos como entrevistas, observación directa de procesos, análisis de tareas, modelado con UML (Unified Modeling Language) y la creación de prototipos funcionales. Según Hull, Jackson y Dick (2011), “comprender el entorno, las metas y las motivaciones de los usuarios resulta tan importante como registrar las funciones que el sistema debe realizar” (p. 52). Esta afirmación destaca que la definición de requerimientos no puede reducirse a aspectos técnicos, pues depende también del conocimiento del contexto social y organizacional en el que el software operará.

En concordancia con lo anterior, Pressman y Maxim (2020) enfatizan que la definición de requerimientos “no solo documenta lo que el sistema debe hacer, sino que también establece los criterios de aceptación que permitirán verificar su cumplimiento” (p. 138). Así, la claridad y la verificabilidad de los requerimientos se convierten en los pilares sobre los cuales se construye todo el proceso de desarrollo.

El resultado tangible de esta etapa es el Documento de Especificación de Requerimientos de Software (SRS), un informe formal que define con precisión las funciones, restricciones y propiedades del sistema. Dicho documento sirve como guía para el diseño, la programación y las pruebas, garantizando la trazabilidad de cada requerimiento desde su origen hasta su implementación final. Robertson y Robertson (2012) señalan que “la especificación de requerimientos debe ser lo suficientemente estructurada para permitir su validación técnica y lo bastante clara para facilitar su comprensión por parte de los usuarios” (p. 93).

En definitiva, el proceso de definición constituye el núcleo de la ingeniería de requerimientos. Su éxito depende de la capacidad del analista para equilibrar la comprensión humana con la precisión técnica, construyendo un puente sólido entre las necesidades del cliente y la solución tecnológica que el sistema proporcionará.

Errores comunes en la definición de requerimientos

La definición de requerimientos es una de las actividades más críticas del desarrollo de software, pero también una de las más propensas a errores. Estos fallos pueden originarse por la falta de comunicación entre los participantes, la ambigüedad en la redacción o la ausencia de métodos estructurados para la recopilación y análisis de la información. Sommerville (2011) advierte que “la ambigüedad del lenguaje natural, la escasa participación de los usuarios y los cambios no controlados en las necesidades constituyen las principales fuentes de error en la definición de requerimientos” (p. 130).

En efecto, el uso del lenguaje cotidiano suele conducir a interpretaciones diversas. Un requerimiento formulado de manera vaga como “el sistema debe ser rápido y confiable” puede parecer comprensible, pero resulta imposible de verificar objetivamente. Pressman y Maxim (2020) señalan que “las descripciones imprecisas o subjetivas provocan inconsistencias y fallas de comunicación que afectan el diseño y la implementación del sistema” (p. 142). Este tipo de ambigüedad no solo retrasa el desarrollo, sino que incrementa los costos al requerir correcciones en etapas avanzadas del proyecto.

Otro error frecuente consiste en asumir que los usuarios comprenden plenamente sus propias necesidades. Sommerville (2011) sostiene que, en muchos casos, “los usuarios no pueden articular de forma completa lo que esperan del sistema hasta que observan una versión funcional o un prototipo” (p. 124). Por esta razón, la interacción temprana con el cliente y el empleo de técnicas de prototipado se vuelven esenciales para refinar y validar los requerimientos antes de formalizarlos.

De igual modo, la omisión de requerimientos no funcionales representa una causa frecuente de problemas. Estos requerimientos relacionados con el rendimiento, la seguridad, la disponibilidad o la usabilidad suelen ser subestimados, aunque son determinantes para la calidad global del software. Hull, Jackson y Dick (2011) subrayan que “la ausencia de

especificaciones claras sobre las propiedades de calidad del sistema puede conducir a soluciones técnicamente correctas pero inaceptables para el usuario” (p. 59).

“Los errores en los requerimientos son los más costosos de corregir, ya que su impacto se multiplica en las fases posteriores del desarrollo.”

(Pressman, 2010, p. 140)

Esta observación sintetiza una realidad ampliamente documentada: los defectos en los requerimientos se propagan a lo largo del ciclo de vida del software, afectando tanto la planificación como las pruebas y el mantenimiento. Según Cristiá (2018), más del 40% de los fallos en proyectos de software tienen su origen en una definición deficiente de los requerimientos, lo que demuestra la importancia de abordar esta fase con rigor metodológico y comunicación constante entre las partes.

Asimismo, la falta de gestión de cambios es otro de los errores más comunes. Los proyectos de software están expuestos a modificaciones inevitables debido a la evolución de las necesidades del negocio o a nuevas restricciones técnicas. Pohl (2010) explica que “la incapacidad para manejar el cambio de forma estructurada conduce a la pérdida de trazabilidad y a inconsistencias en la implementación” (p. 152). Para evitarlo, es necesario establecer un proceso formal de control de versiones y revisión periódica de los requerimientos.

Como resumen Robertson y Robertson (2012), “los fallos más graves no provienen de la complejidad técnica, sino de los malentendidos humanos sobre lo que realmente se necesita” (p. 70). En este sentido, la claridad, la comunicación y la colaboración interdisciplinaria son elementos esenciales para prevenir ambigüedades y asegurar que el sistema desarrollado cumpla con su propósito.

Dificultades esenciales y accidentales

“La parte más difícil de construir un sistema de software es decidir con precisión qué construir. Ninguna otra parte del trabajo conceptual es tan difícil como establecer los requerimientos técnicos detallados... Ninguna otra parte del trabajo puede dañar tanto el sistema resultante si se hace mal. Ninguna otra parte es tan difícil de corregir después.” [Brooks, 87]

Incluso nuestros objetivos más detallados parecen razonablemente sencillos; ¿por qué, entonces, tantos esfuerzos de desarrollo no logran alcanzarlos?

La respuesta corta es que la satisfacción mutua de estos objetivos, en la práctica, es inherentemente difícil. Para entender por qué, es útil reflexionar sobre algunos puntos planteados por el Dr. Brooks [Brooks, 87] acerca de por qué la ingeniería de software es difícil y sobre la distinción que él hace entre las dificultades esenciales —aquellas inherentes al problema— y las dificultades accidentales —aquellas introducidas por una práctica imperfecta—.

Aunque los requerimientos son inherentemente difíciles, no hay duda de que estas dificultades se multiplican muchas veces debido a las deficiencias de las prácticas actuales.

Dificultades esenciales

Las siguientes **dificultades esenciales** acompañan a cada uno de los objetivos de los requerimientos:

- Comprensión

Las personas no saben lo que quieren.

Esto no significa que las personas no tengan una idea general de para qué es el software.

Más bien, no comienzan con una comprensión precisa y detallada de qué funciones deben estar en el software, cuál debe ser la salida para cada posible entrada, cuánto tiempo debe tomar cada operación, cómo una decisión afectará a otra, y así sucesivamente.

- Comunicación

Los requerimientos de software son difíciles de comunicar eficazmente.

Como señala el Dr. Brooks, las estructuras conceptuales de los sistemas de software son complejas, arbitrarias y difíciles de visualizar.

La dificultad inherente de la comunicación se ve agravada por la diversidad de propósitos y audiencias de una especificación de requerimientos.

Tal grado de comunalidad típicamente no se da entre los muchos grupos diversos (por ejemplo, clientes, ingenieros de sistemas, gerentes) que deben usar una especificación de requerimientos de software.

- Control

También existen dificultades inherentes al control del desarrollo del software.

La naturaleza arbitraria e invisible del software hace difícil anticipar qué requerimientos se cumplirán fácilmente y cuáles podrían diezmar el presupuesto y el cronograma del proyecto, si es que pueden cumplirse en absoluto.

La baja fidelidad de la planificación del software se ha convertido en un cliché, sin embargo, los requerimientos son la única base disponible para planificar o hacer un seguimiento de un plan. Esta situación se ve inmensamente agravada por la maleabilidad inherente del software. Para muchos desarrolladores industriales, la gestión del cambio es el problema más crítico en los requerimientos.

- Preocupaciones inseparables

Al buscar soluciones a los problemas anteriores, nos enfrentamos a una dificultad adicional: los temas no pueden separarse fácilmente ni abordarse por partes.

Por ejemplo, los desarrolladores han intentado resolver el problema de los requerimientos cambiantes congelando los requerimientos antes de que comience el diseño.

Esto resulta impráctico debido al problema de comprensión: el cliente no sabrá completamente lo que quiere hasta que lo vea.

Dificultades accidentales:

Se usa el término “accidental” en contraste con “esencial”, no para implicar que las dificultades surjan por casualidad, sino porque son el producto de fallas comunes en la gestión, la elicitación, la especificación o el uso de los requerimientos.

- Escritos como una idea tardía

Sigue siendo una práctica común que la documentación de requerimientos se desarrolle solo después de que el software ya ha sido escrito. En muchos proyectos, la tentación de apresurarse hacia la implementación antes de comprender adecuadamente los requerimientos resulta irresistible.

Esta documentación posterior a los hechos inevitablemente viola el objetivo básico de definir lo que el sistema debe hacer en lugar de cómo debe hacerlo, ya que es simplemente una especificación del código tal como se escribió. Se produce después de los hechos, por lo que

no se planifica ni se gestiona como una parte esencial del desarrollo, sino que se arma apresuradamente.

- Confusión en el propósito

Debido a que existen tantos públicos potenciales para una especificación de requerimientos, con diferentes puntos de vista, el propósito exacto del documento se vuelve confuso.

- Se utiliza para vender el producto al cliente, por lo que incluye promociones de marketing exaltando las virtudes del producto.
- Es la única documentación de lo que hace el sistema, por lo que incluye material introductorio, explicativo y de descripción general.
- Es un documento contractual, por lo que se redacta de manera intencionadamente imprecisa para permitir al desarrollador cierta libertad en el producto entregado, o al cliente cierta flexibilidad para hacer cambios sin costo.
- Es el vehículo de comunicación de las decisiones sobre los detalles del software a los diseñadores y programadores, por lo que incorpora elementos de diseño e implementación.

El resultado es un documento en el que no está claro cuáles afirmaciones representan verdaderos requerimientos y cuáles deberían asignarse más propiamente al marketing, diseño u otra documentación.

- No diseñado para ser útil

A menudo, en la prisa por llegar a la implementación, se dedica muy poco esfuerzo a los requerimientos. La especificación de requerimientos no se espera que sea útil, y de hecho, esto se convierte en una profecía autocumplida.

Como el documento no se considera útil, se dedica poco esfuerzo a diseñarlo, escribirlo, revisarlo o gestionar su creación y evolución. El resultado más evidente es una pobre organización.

- Carece de propiedades importantes

La falta de previsión, la confusión de propósito, o la falta de diseño y ejecución cuidadosos conducen a requerimientos que carecen de propiedades críticas para una buena especificación técnica.

Los requerimientos, si se documentan en absoluto, resultan ser redundantes, inconsistentes, incompletos, imprecisos e inexactos.

Buenas prácticas para una definición exitosa de requerimientos de software

Según Cristiá (2016, p. 8), una de las claves más importantes para una buena definición de requerimientos es **escuchar más que hablar** durante las entrevistas iniciales con los usuarios. El autor enfatiza que el ingeniero de requerimientos debe actuar como un intérprete entre el lenguaje cotidiano del cliente y el lenguaje técnico del software. Escuchar activamente permite identificar las verdaderas necesidades del usuario, más allá de sus primeras expresiones o deseos. Asimismo, Berezin (1999, p. 2) resalta que el entrevistador debe promover un diálogo abierto y hacer preguntas aclaratorias, evitando imponer soluciones antes de comprender el problema. En otras palabras, la comunicación efectiva comienza con la escucha empática y la validación constante de lo que el cliente expresa.

Por otro lado, Devanbu (2000, p. 3) señala que **el uso de prototipos o diagramas es una herramienta esencial** para visualizar las ideas del usuario y reducir malentendidos. Muchos clientes no pueden imaginar un sistema a partir de descripciones verbales, por lo que las representaciones gráficas —como modelos de casos de uso, diagramas de flujo o pantallas simuladas— permiten validar tempranamente la interpretación de los requerimientos. De manera similar, el manual de la Universidad Continental (Wong Durand, 2017, p. 57) indica que los prototipos ayudan a “traducir” los requerimientos no técnicos a un lenguaje visual accesible, lo cual facilita la definición consensuada del sistema.

Además, Cristiá (2016, p. 10) subraya que cada requerimiento debe **redactarse de forma clara, breve y verificable**. Esto significa que todo enunciado debe poder ser probado o validado, evitando términos ambiguos como “rápido” o “fácil de usar”. Berezin (1999, p. 5) coincide en que una redacción uniforme y simple mejora la comprensión entre todos los actores del proyecto. De hecho, recomienda emplear un formato estándar para cada requerimiento, incluyendo un identificador, una descripción precisa y los criterios de aceptación. Según la

Universidad Continental (Wong Durand, 2017, p. 63), esta práctica evita contradicciones y asegura la trazabilidad posterior durante la validación del software.

La revisión y aprobación formal del documento también son aspectos fundamentales. Devanbu (2000, p. 4) explica que **la definición de requerimientos no finaliza con la redacción**, sino con su validación y aprobación por todas las partes interesadas. Esta revisión debe realizarse de manera estructurada, identificando ambigüedades, errores o inconsistencias. Cristiá (2016, p. 11) destaca que la participación activa del cliente en la revisión garantiza compromiso y reduce conflictos futuros. Así, la aprobación formal de cada versión del documento genera una línea base confiable sobre la cual se desarrolla todo el proyecto.

Otro elemento clave es **mantener la trazabilidad entre los requerimientos, el diseño y las pruebas**. Según Wong Durand (2017, p. 70), cada requerimiento debe poder rastrearse desde su origen hasta su implementación y verificación final, permitiendo gestionar los cambios de forma controlada. Devanbu (2000, p. 5) sostiene que esta práctica asegura coherencia y evita que se pierda el sentido del requerimiento original durante el desarrollo. Asimismo, Cristiá (2016, p. 13) indica que una buena trazabilidad no solo facilita la gestión del cambio, sino que también contribuye a mejorar la calidad del software y la satisfacción del usuario.

Finalmente, Berezin (1999, p. 6) recomienda **utilizar estándares y plantillas reconocidas internacionalmente**, como la estructura del IEEE 830, para asegurar consistencia y profesionalismo en la documentación. Dicho estándar establece secciones básicas para todo documento de requerimientos: introducción, descripción general, requerimientos específicos y anexos. De acuerdo con Cristiá (2016, p. 15), adoptar formatos estructurados fomenta la comprensión, facilita la revisión y estandariza el trabajo entre equipos. La Universidad Continental (Wong Durand, 2017, p. 83) también sugiere el uso de herramientas de gestión de requerimientos (como Jira o Trello) o plantillas en Word o Excel para mantener actualizada y accesible la información a lo largo del ciclo de vida del proyecto.

En conclusión, las buenas prácticas para una definición exitosa de requerimientos se centran en **la comunicación efectiva, la claridad documental, la validación continua y la trazabilidad del proceso**. Tal como señalan las distintas fuentes consultadas, definir correctamente los requerimientos desde el inicio no solo evita costos futuros, sino que constituye la base del éxito de todo proyecto de software.

4. Ejemplos

EJEMPLO 1 — Sistema hospitalario (Registro de pacientes)

Fuente:

Cristiá, M. (2018). *Introducción a la Ingeniería de Requerimientos*. Universidad Nacional de Rosario, Argentina. Recuperado de <https://www.fceia.unr.edu.ar/~mcristia/publicaciones/ingreq-a.pdf>, p. 15

Contexto:

Un hospital regional busca desarrollar un sistema informático que registre la atención de pacientes y genere reportes automáticos por servicio médico, para reducir errores en el manejo de historiales clínicos.

Problema:

El documento inicial del proyecto solo mencionaba: “El sistema debe registrar pacientes”. La falta de detalle provocó confusión sobre qué información se debía almacenar y cómo debía generarse el reporte médico.

Aplicación del concepto:

Según Cristiá (2018, p. 15), un requerimiento de software es una *condición o capacidad que debe poseer el sistema para satisfacer una necesidad o cumplir una norma específica*.

El requerimiento se redefine así:

“El sistema debe permitir registrar nombre completo, edad, número de expediente, diagnóstico y área médica de cada paciente, para generar reportes diarios de atención por servicio.”

Resultado:

El requerimiento se vuelve medible y verificable, evitando ambigüedades y mejorando la comunicación entre analistas, desarrolladores y personal médico.

EJEMPLO 2 — Comercio electrónico (Proceso de pago en línea)

Fuente:

Berry, D. M. (2004). *Software Requirements: A Tutorial*. University of California, Davis. Recuperado de <https://www.cs.ucdavis.edu/~devanbu/teaching/160/docs/req-paper.pdf>, p. 4

Contexto:

Una tienda en línea desea mejorar el flujo de compra de sus clientes, ya que muchos abandonan el carrito debido a errores en la pasarela de pago o a una experiencia confusa.

Problema:

El requerimiento original decía: “El sistema debe permitir comprar productos en línea”. Esta formulación era demasiado general y no especificaba pasos, validaciones o confirmaciones.

Aplicación del concepto:

Berry (2004, p. 4) destaca que los requerimientos deben ser *específicos, claros y relacionados directamente con las necesidades del usuario final*.

Con base en ello, se redefine el requerimiento:

“El sistema permitirá al usuario seleccionar productos, agregarlos al carrito, elegir método de pago (tarjeta o PayPal) y completar la compra con confirmación electrónica del pedido.”

Resultado:

Esta versión establece un flujo de compra verificable, mejora la comunicación entre las áreas de desarrollo y diseño, y permite probar la experiencia del usuario mediante criterios medibles.

EJEMPLO 3 — Aplicación universitaria (Consulta de calificaciones)**Fuente:**

Wong Durand, S. (2017). *Análisis y requerimientos de software*. Universidad Continental. Recuperado de https://repositorio.continental.edu.pe/bitstream/20.500.12394/4281/1/DO_FIN_103_MAI_UC093_9_2018.pdf, pp. 21–22

Contexto:

Una universidad desea desarrollar una aplicación móvil que permita a los estudiantes consultar sus calificaciones y horarios de clases de forma segura y rápida.

Problema:

El requerimiento inicial era ambiguo: “La aplicación mostrará información académica”. No se indicaba qué tipo de información, con qué frecuencia se actualizaba, ni las medidas de seguridad necesarias.

Aplicación del concepto:

Wong Durand (2017, pp. 21–22) indica que los requerimientos deben clasificarse correctamente para facilitar su diseño e implementación. Se propone la siguiente definición:

- **Requerimiento funcional:** “La aplicación permitirá al estudiante consultar sus calificaciones y horarios por semestre.”
- **Requerimiento no funcional:** “La aplicación deberá cargar los datos en menos de 2 segundos y utilizar conexión segura mediante protocolo HTTPS.”

Resultado:

Esta distinción permite mayor precisión en la documentación del sistema y garantiza que tanto programadores como usuarios finales comprendan las expectativas funcionales y de rendimiento del software.

EJEMPLO 4 — Banca en línea (Autenticación de usuarios)

Fuente: Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico*. McGraw-Hill Interamericana, p. 102

Contexto:

Un banco desarrolla una plataforma web para operaciones de clientes.

Problema:

El requerimiento “El sistema debe permitir iniciar sesión” no define seguridad ni validaciones.

Aplicación del concepto:

Pressman (2010) señala que los requerimientos deben describir el comportamiento esperado bajo condiciones específicas.

Redefinición:

“El sistema permitirá a los usuarios autenticarse con nombre de usuario, contraseña y código OTP enviado por SMS.”

Resultado:

Se mejora la seguridad y la trazabilidad del acceso, cumpliendo con normas de autenticación reforzada.

EJEMPLO 5 — Gobierno digital (Gestión de trámites)

Fuente: Sommerville, I. (2011). *Ingeniería de software* (9ª ed.). Pearson Educación, p. 105

Contexto:

Un municipio necesita implementar un portal para que los ciudadanos puedan realizar trámites en línea.

Problema:

El requerimiento “El sistema debe permitir realizar trámites” es demasiado general.

Aplicación del concepto:

Sommerville (2011) explica que los requerimientos funcionales deben detallar *qué servicios ofrece el sistema al usuario*.

Nuevo requerimiento:

“El sistema permitirá a los ciudadanos solicitar actas de nacimiento, pagar derechos y descargar comprobantes electrónicos desde su perfil personal.”

Resultado:

El requerimiento se vuelve comprensible para desarrolladores y usuarios gubernamentales, de esta forma se puede desarrollar de manera correcta el proyecto

EJEMPLO 6 — Salud pública (Control de vacunación)

Fuente: Pohl, K. (2010). *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer-Verlag. <https://doi.org/10.1007/978-3-642-12578-2>, p. 88

Contexto:

Un sistema nacional de salud busca llevar control digital de vacunación infantil.

Problema:

El requerimiento “El sistema debe registrar vacunas” no define periodicidad ni validación.

Aplicación del concepto:

Pohl (2010) indica que los requerimientos deben ser *completos y consistentes*.

Reescritura:

“El sistema debe permitir registrar dosis aplicadas, fecha, lote de vacuna y nombre del médico, validando automáticamente los intervalos entre dosis.”

Resultado:

La definición clara reduce errores en registros y facilita auditorías sanitarias.

EJEMPLO 7 — Transporte urbano (Gestión de rutas)

Fuente: Robertson, S., & Robertson, J. (2012). *Mastering the Requirements Process*. Addison-Wesley, p. 46

Contexto:

Una empresa de transporte busca optimizar rutas y horarios de autobuses mediante una aplicación.

Problema:

El requerimiento “El sistema mostrará rutas” es incompleto y ambiguo.

Aplicación del concepto:

Robertson y Robertson (2012) destacan que los requerimientos deben enfocarse en *qué necesita el usuario, no en cómo se implementa*.

Nuevo requerimiento:

“El sistema mostrará rutas disponibles según origen y destino seleccionados, indicando horarios y tiempo estimado de llegada.”

Resultado:

Se mejora la utilidad del sistema y la experiencia del usuario.

EJEMPLO 8 — Manufactura (Control de inventarios)

Fuente: Hull, E., Jackson, K., & Dick, J. (2011). *Requirements Engineering (3rd ed.)*. Springer. <https://doi.org/10.1007/978-1-84996-405-0>, p. 63

Contexto:

Una fábrica necesita un sistema que controle inventarios y alertas de reposición.

Problema:

El requerimiento “El sistema manejará inventarios” no especifica variables ni acciones automáticas.

Aplicación del concepto:

Hull et al. (2011) indican que los requerimientos deben describir condiciones de entrada, procesamiento y salida.

Redefinición:

“El sistema deberá registrar productos, cantidades y ubicación; generará alertas automáticas cuando un ítem llegue al 10% del stock mínimo.”

Resultado:

El requerimiento permite desarrollar una función verificable y alineada con objetivos productivos.

EJEMPLO 9 — Redes sociales (Publicaciones y privacidad)

Fuente: Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education, p. 115

Contexto:

Una startup diseña una red social enfocada en compartir publicaciones con controles de privacidad.

Problema:

El requerimiento “El sistema permitirá publicar contenido” no define permisos ni visibilidad.

Aplicación del concepto:

Pressman y Maxim (2020) destacan la importancia de requerimientos medibles y centrados en

la interacción usuario-sistema.

Definición:

“El sistema permitirá crear publicaciones con texto o imagen, configurando su visibilidad como pública, privada o solo para amigos.”

Resultado:

El requerimiento se convierte en funcional, verificable y alineado con políticas de privacidad.

EJEMPLO 10 — Energía (Medición inteligente)

Fuente: IEEE Std 830-1998. *Recommended Practice for Software Requirements Specifications*. IEEE Computer Society, p. 6

Contexto:

Una empresa eléctrica implementa medidores inteligentes para monitorear el consumo en tiempo real.

Problema:

El requerimiento “El sistema medirá el consumo” no define precisión, frecuencia ni formato.

Aplicación del concepto:

Según IEEE 830-1998, los requerimientos deben ser *medibles, verificables y rastreables*.

Nuevo requerimiento:

“El sistema deberá registrar el consumo energético de cada cliente cada 15 minutos, con precisión de ± 2 %, y enviar los datos en formato XML al servidor central.”

Resultado:

La definición cumple estándares de calidad y facilita la validación técnica.

5. Video

Adjunto en la página del tutorial

6. Software interactivo

Palabras a buscar en una sopa de letras en el tutorial:

REQUERIMIENTO	ANÁLISIS	RESTRICCIÓN
SOFTWARE	ESPECIFICACIÓN	DESCRIPCIÓN
USUARIO	VALIDACIÓN	OBJETIVO
CLIENTE	VERIFICACIÓN	PROCESO
NECESIDAD	FUNCIONAL	SATISFACCIÓN
SISTEMA	NOFUNCIONAL	

7. Cuestionario interactivo

Se encuentra en otro documento adjunto.

8. Conclusión

La definición y gestión adecuada de los requerimientos de software constituye uno de los pilares fundamentales para el éxito de cualquier proyecto de desarrollo. A lo largo del trabajo se ha demostrado que los requerimientos no solo representan las necesidades funcionales y no funcionales del sistema, sino que también actúan como un puente comunicativo entre el cliente, los usuarios y el equipo técnico. Su correcta formulación garantiza que el producto final cumpla con las expectativas y objetivos establecidos desde el inicio.

Las dificultades esenciales, como la comprensión y comunicación de las necesidades, así como las dificultades accidentales derivadas de prácticas inadecuadas, evidencian que el reto no reside únicamente en la complejidad técnica, sino también en los procesos humanos y organizacionales involucrados. Por ello, la ingeniería de requerimientos debe abordarse de manera sistemática, utilizando metodologías, estándares y herramientas que promuevan la claridad, la verificabilidad y la trazabilidad.

Asimismo, la aplicación de buenas prácticas, como la escucha activa, el uso de prototipos, la redacción uniforme, la revisión formal y la gestión de versiones, permite reducir los errores más comunes y mejorar la calidad del software desarrollado. En conclusión, comprender, documentar y validar los requerimientos no es una etapa más del ciclo de vida del software, sino el fundamento sobre el cual se construye la calidad, la funcionalidad y la satisfacción del usuario final.

9. Bibliografía

Berezin, T. (1999). Writing a Software Requirements Document. Adelphi University. Disponible en <https://home.adelphi.edu/~siegfried/cs480/ReqsDoc.pdf>

Berry, D. M. (2004). Software Requirements: A Tutorial. U.C. Davis. Recuperado de <https://www.cs.ucdavis.edu/~devanbu/teaching/160/docs/req-paper.pdf>

Cristiá, M. (2018). Introducción a la Ingeniería de Requerimientos. Recuperado de <https://www.fceia.unr.edu.ar/~mcristia/publicaciones/ingreq-a.pdf>

Devanbu, P. (2000). Software Requirements: A Tutorial. University of California, Davis. Disponible en <https://www.cs.ucdavis.edu/~devanbu/teaching/160/docs/req-paper.pdf>

Hull, E., Jackson, K., & Dick, J. (2011). Requirements Engineering (3rd ed.). Springer. <https://doi.org/10.1007/978-1-84996-405-0>

IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications

International Institute of Business Analysis [IIBA]. (2009). A Guide to the Business Analysis Body of Knowledge (BABOK® Guide) (Version 2.0). IIBA.

Pohl, K. (2010). Requirements Engineering: Fundamentals, Principles, and Techniques. Springer-Verlag. <https://doi.org/10.1007/978-3-642-12578-2>

Pressman, R. S. (2002). Ingeniería del software: Un enfoque práctico (5ª ed.). McGraw-Hill Interamericana.

Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico (7ª ed.). McGraw-Hill Interamericana.

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

Robertson, S., & Robertson, J. (2012). Mastering the Requirements Process: Getting Requirements Right (3rd ed.). Addison-Wesley.

Rodríguez, C. T. (2017). Impacto de los requerimientos en la calidad de software. TIA, Vol. 5 No. 2, 162-174. Recuperado de <https://revistas.udistrital.edu.co/index.php/tia/article/download/7607/pdf/60871>

Sommerville, I. (2011). Ingeniería de software (9ª ed.). Pearson Educación.

Wong Durand, A. (2017). Ingeniería de software: Fundamentos, métodos y estándares. Alfaomega Grupo Editor.

Wong Durand, S. (2017). Análisis y requerimientos de software. Repositorio Continental.
Recuperado de
https://repositorio.continental.edu.pe/bitstream/20.500.12394/4281/1/DO_FIN_103_MAI_UC0939_2018.pdf